

Vibe Coding

A practical process for turning an idea into a polished, production-ready website or app - with checklists, guardrails, prompts, and launch discipline.

OPERATING PRINCIPLE You own product intent and decisions. The coding agent accelerates research, implementation, iteration, and debugging. Every meaningful change is planned, verified, and reviewed.

WHAT THIS PLAYBOOK COVERS

STAGE	OUTCOME	PRIMARY ARTIFACTS
Think	Problem clarity	Problem statement, success definition
Define	Scope clarity	Product brief, acceptance criteria, roadmap
Design	Visual source of truth	Screens, tokens, responsive rules
Prepare	Agent-ready context	AGENTS.md, docs, skills, repo
Build	Working slices	Components, data, features
Integrate	Real services	APIs, auth, storage, webhooks
Verify	Release confidence	Lint, typecheck, tests, QA, security
Launch	Production readiness	Domain, SEO, monitoring, rollback
Iterate	Controlled change	Issues, decisions, metrics, releases

PROFESSIONAL RULE Never let the agent silently widen scope. One prompt should normally produce one small, testable outcome.

01 The Professional Vibe Coding Loop

Use this loop for almost every meaningful feature. The loop is more important than any specific IDE, model, library, or framework.

PHASE	HUMAN	AGENT	EXIT CONDITION
1. Context	Point to source-of-truth docs/files.	Read rules and relevant code.	Agent can summarize the task and constraints.
2. Plan	Confirm approach and scope.	Propose smallest implementation.	Plan matches requirements.
3. Implement	Approve the slice.	Edit only necessary files.	Feature exists locally.
4. Verify	Run the changed flow.	Fix lint/type/runtime issues.	Checks pass.
5. Review	Inspect screenshot + diff.	Self-review for regressions.	No unintended changes.
6. Ship	Commit and preview.	Prepare focused commit if asked.	Preview behaves as expected.
7. Learn	Capture feedback/decisions.	Update code/docs where needed.	Next slice starts from a stable state.

DEFAULT SEQUENCE Plan -> Implement -> Lint -> Typecheck -> Test -> Visual QA -> Diff Review -> Commit -> Preview -> Production.

Definition of Done

- ☐ Requirement is implemented exactly as scoped.
- ☐ Responsive behavior works at target breakpoints.
- ☐ Success, empty, loading, error, and disabled states are considered.
- ☐ Accessibility basics are checked.
- ☐ Security and data handling are checked where relevant.
- ☐ Lint, typecheck, tests, and production build pass.
- ☐ Git diff contains only intended changes.
- ☐ Docs/decisions are updated when behavior or architecture changed.

02 Idea -> Product Brief

Start before opening the coding agent. A small, clear brief prevents large rewrites later.

- ☐ Write the problem in one sentence.
- ☐ Define the primary user and the job they need done.
- ☐ Define the desired outcome and how success will be recognized.
- ☐ List the smallest useful version (MVP).
- ☐ Separate must-have, should-have, later, and out-of-scope.

- ☐ Collect references, screenshots, copy examples, and interaction inspiration.
- ☐ Identify facts that must be accurate and things the agent must never invent.
- ☐ Separate static content from dynamic data.
- ☐ Choose the deployment target and constraints early.

Mini brief template

```
PROJECT: [name]
PROBLEM: [what needs to improve]
USER: [primary audience]
OUTCOME: [what success looks like]
MVP: [smallest useful scope]
OUT OF SCOPE: [explicit exclusions]
REFERENCES: [links/files]
CONSTRAINTS: [brand/tech/performance/security]
SUCCESS CHECK: [observable acceptance criteria]
```

Feature request template

```
FEATURE:
PROBLEM:
USER:
DESIRED BEHAVIOR:
ACCEPTANCE CRITERIA:
DEPENDENCIES:
SECURITY CONSIDERATIONS:
PERFORMANCE CONSIDERATIONS:
OUT OF SCOPE:
```

03 Design First, Then Code

- ☐ Create the visual direction before implementation.
- ☐ Capture desktop, tablet, and mobile states.
- ☐ Capture light/dark states when themes are required.
- ☐ Define typography, spacing, colors, radii, borders, shadows, surfaces, and motion.
- ☐ Define hover, focus, active, disabled, loading, empty, error, and success states.
- ☐ Decide what is reusable and what is intentionally one-off.
- ☐ Store screenshots/reference exports in a clear design-reference area.

DESIGN-SYSTEM.md checklist

TOPIC	DEFINE
Source of truth	Screens, design files, approved references
Layout	Containers, max widths, breakpoints, grids, alignment
Typography	Families, sizes, weights, line heights
Color	Light/dark tokens and semantic roles
Shape	Radius, borders, shadows, surfaces
Components	Buttons, badges, cards, forms, navigation, states
Motion	Duration, easing, reduced-motion behavior

TOPIC	DEFINE
Responsive	What changes at each breakpoint
Accessibility	Contrast, focus, keyboard, semantics

SOURCE-OF-TRUTH RULE The design reference is the visual source of truth. Do not invent a different UI unless the product owner explicitly requests it.

04 Project Documentation + Agent Context

A professional Vibe Coding setup gives the agent a compact operating manual. Keep docs operational, current, and easy to scan.

FILE	PURPOSE	WHAT BELONGS INSIDE
AGENTS.md	Agent operating rules	Stack, design authority, architecture rules, safety, workflow, validation, do-not list
PRD.md	Product requirements	Users, goals, scope, non-goals, requirements, success criteria
DESIGN-SYSTEM.md	Visual rules	Tokens, responsive behavior, components, interaction states
ARCHITECTURE.md	System structure	Boundaries, data flow, ownership, server/client rules, tradeoffs
SECURITY.md	Security baseline	Secrets, validation, auth, access control, webhooks, logging
CODE-STYLE.md	Coding conventions	TypeScript, naming, imports, components, comments, testing style
DATABASE.md	Persistence plan	When a DB is needed, schema approach, migrations, retention, access
API-GUIDE.md	Integration rules	External APIs, normalization, caching, retries, errors, limits
ROADMAP.md	Evolution plan	Milestones, priorities, deferred ideas, release themes
KNOWN-ISSUES.md	Known limitations	Issue, impact, workaround, priority, status
docs/decisions/*	Architecture decisions	Context, options, chosen decision, rationale, tradeoffs
docs/AI-DECISIONS.md	AI-specific decisions	Prompts/decisions that establish durable project behavior

FILE	PURPOSE	WHAT BELONGS INSIDE
docs/COMPONENTS.md	Component inventory	Purpose, props, variants, usage, dependencies
docs/ROUTES.md	Route map	Purpose, access, data, SEO, loading/error states

AGENTS.md minimum structure

- ☐ Project context and current goal.
- ☐ Non-negotiable design rules.
- ☐ Architecture and server/client rules.
- ☐ Coding standards.
- ☐ Security requirements.
- ☐ Performance and accessibility requirements.
- ☐ Validation commands.
- ☐ Git safety rules.
- ☐ Decision priority when instructions conflict.
- ☐ Explicit do-not list.

05 Skills, Tools, Repository & Environment

Use skills as reusable guidance, not as a substitute for architecture. Install only capabilities that materially help the current project.

Skills checklist

- ☐ List the capabilities your coding-agent platform supports.
- ☐ Install the base framework skill.
- ☐ Install the UI/component skill when using a component system.
- ☐ Install design/web-guideline skills for visual and accessibility quality.
- ☐ Install performance guidance.
- ☐ Add testing/security/deployment guidance when the project needs it.
- ☐ Keep a record of installed/allowed skills.
- ☐ Read the relevant skill instructions before implementation.
- ☐ Avoid installing large collections “just in case.”

Repository checklist

- ☐ Initialize the framework with TypeScript and the chosen lint/build tools.
- ☐ Create a clean folder structure.
- ☐ Create .env.example and secret conventions.
- ☐ Add lint, typecheck, test, and build commands.
- ☐ Add a README with setup and deployment notes.
- ☐ Commit the clean baseline before major changes.
- ☐ Verify a fresh clone can install and run the project.

```

app/
components/
  ui/
    [feature]/
content/
lib/
public/
docs/
tests/
AGENTS.md
README.md
.env.example

```

DEPENDENCY GATE Before installing a package ask: Do I need it? Can existing tools solve it? Is it maintained? What does it add to bundle size, complexity, or risk?

06 The Vibe Coding Build Method

Prompt formula

```

CONTEXT: [docs + relevant files]
GOAL: [single outcome]
SOURCE OF TRUTH: [file/design reference]
REQUIREMENTS: [specific behavior]
CONSTRAINTS: [what must not change]
ACCEPTANCE: [observable checks]
VALIDATION: [lint/typecheck/tests/build]
STOP: [do not expand scope]

```

Prompt checklist

- ☐ State exactly what should change.
- ☐ State exactly what must not change.
- ☐ Point to source-of-truth files.
- ☐ Reuse existing components/data where possible.
- ☐ Ask the agent to inspect before editing when the root cause is uncertain.
- ☐ Use one task per prompt whenever practical.
- ☐ Require validation commands.
- ☐ Give a clear stop condition.

Professional AI context strategy

DO	AVOID
Give only relevant docs/files.	Dump the whole repository into every prompt.
State protected areas.	Assume the agent will infer what not to touch.
Reference stable decisions.	Re-open already settled architecture every time.
Work in small slices.	Ask for the entire product in one prompt.
Ask for evidence when uncertain.	Let the agent guess and rewrite broadly.

STOP CONDITION A good prompt ends with a boundary: implement this feature, verify it, and stop. Do not refactor

unrelated code.

07 Content, Components & State Design

- ☐ Keep content/data in typed files separate from rendering.
- ☐ Keep reusable UI primitives in components/ui.
- ☐ Keep feature composition in domain folders.
- ☐ Keep integration logic behind lib/service modules.
- ☐ Default to Server Components; use Client Components only for interaction, state, or browser APIs.
- ☐ Model optional fields explicitly (logos, links, dates, tags, status).
- ☐ Do not fabricate profile, company, product, metric, or customer facts.

State matrix

STATE	ASK
Default	What does the user see first?
Loading	What is shown while data is arriving?
Success	What confirms the action worked?
Empty	What if there is no data yet?
Error	What does the user see and what can they do next?
Disabled	When is interaction intentionally unavailable?
Offline/unavailable	What happens when a dependency fails?
Permission denied	What if the user lacks access?

When to abstract

- Repeated markup/data shape appears 2-3+ times: consider a reusable component/model.
- One component has unique behavior: keep it local.
- Feature needs browser state: isolate a Client Component.
- External integration: hide it behind a service/lib boundary.
- No current reuse need: do not build a generic framework.

08 Integrations, APIs, Data & Webhooks

- ☐ Write down the external service and why it is needed.
- ☐ Create a small integration module instead of scattering fetch logic.
- ☐ Keep credentials server-side.
- ☐ Normalize external responses into your own types.
- ☐ Handle timeouts, retries, rate limits, errors, empty data, and unavailable services.

- ☐ Use caching/revalidation where appropriate.
- ☐ Validate untrusted external data at runtime.
- ☐ Log safely; never log secrets or sensitive payloads.
- ☐ Add fallbacks when non-critical integrations fail.
- ☐ Verify webhook signatures and protect replay-sensitive operations.

Data boundary checklist

LAYER	RULE
UI	Consume typed, UI-ready data; do not embed credentials.
API/service	Validate, normalize, cache, retry, and handle failures.
Database	Use constraints, least privilege, migrations, retention.
Webhook	Verify signature, validate payload, make handlers idempotent.
Logs	Record useful diagnostics without secrets or private data.

09 Architecture Decisions & Change Records

Professional work records why important choices were made so future humans and agents do not accidentally reverse them.

```
docs/decisions/
  001-[decision].md
  002-[decision].md
# Decision: [title]
Context: [why a decision was needed]
Options: [real alternatives]
Chosen: [selected approach]
Why: [rationale]
Tradeoffs: [costs/risks]
Consequences: [what changes now]
Status: [active/superseded]
```

AI decision log

- ☐ Record durable AI-driven decisions that affect architecture, naming, patterns, or constraints.
- ☐ Reference the files or prompt that established the decision.
- ☐ Mark decisions superseded when the project intentionally changes direction.

10 Quality Gates: Lint, Typecheck, Tests & Visual QA

Compilation success is not release readiness. Use multiple gates.

GATE	MINIMUM CHECK	WHEN
Lint	No lint errors/warnings that violate project policy	Every meaningful batch
Typecheck	TypeScript passes without type errors	Every meaningful batch

GATE	MINIMUM CHECK	WHEN
Unit/integration	Critical logic and boundaries behave correctly	Risky or reusable logic
E2E	Important user flows work end-to-end	Pre-release and after major changes
Visual QA	Screenshots match design and responsive intent	Every visual change
Accessibility	Keyboard, focus, labels, contrast, reduced motion	Every interactive feature
Production build	Build completes in CI/local production mode	Before release

Visual regression checklist

- ☐ Desktop checked.
- ☐ Tablet checked.
- ☐ Mobile checked.
- ☐ Light mode checked.
- ☐ Dark mode checked.
- ☐ No horizontal overflow.
- ☐ No clipping or overlap.
- ☐ Images and logos render correctly.
- ☐ Hover/focus states are intentional.
- ☐ Motion is not required for comprehension.

SCREENSHOT HABIT For any visual change, capture a before/after screenshot. It is faster to compare pixels than to debate whether the UI “feels” different.

11 Security & Threat Modeling

- ☐ Never commit secrets, tokens, private keys, or production credentials.
- ☐ Use environment variables and secret managers.
- ☐ Validate untrusted input at runtime.
- ☐ Enforce authentication and authorization server-side.
- ☐ Keep API errors user-safe; avoid exposing internals.
- ☐ Sanitize/escape user-generated content and protect against XSS.
- ☐ Verify webhook signatures and make sensitive handlers idempotent.
- ☐ Use least-privilege access for databases and services.
- ☐ Review dependencies before major releases.
- ☐ Define retention and deletion behavior for personal/sensitive data.

Threat model prompts

- What can the user/attacker control?
- What data could they access if a check fails?
- What happens if input is malicious?
- What happens if a secret leaks?
- Can a request be replayed?
- What happens if an external dependency lies, fails, or times out?

12 Performance, Accessibility & UX Discipline

Performance budget checklist

- ☐ Optimize large images and avoid shipping oversized assets.
- ☐ Load fonts efficiently and minimize font variants.
- ☐ Avoid unnecessary Client Components and browser JS.
- ☐ Limit third-party scripts.
- ☐ Scope heavy effects such as blur, WebGL, and complex animation.
- ☐ Cache or revalidate remote data appropriately.
- ☐ Measure production performance after major features.

Accessibility checklist

- ☐ Semantic headings and landmarks.
- ☐ Keyboard navigation works.
- ☐ Visible focus states.
- ☐ Accessible names for interactive controls.
- ☐ Alt text where images convey meaning.
- ☐ Sufficient text/background contrast.
- ☐ Reduced-motion behavior.
- ☐ Touch targets are usable on mobile.

DESIGN RESTRAINT Animation, glass, blur, cursor effects, and other polish should clarify or delight without becoming required for the product to function.

13 SEO, Metadata, Launch & Production

- ☐ Set a canonical URL and correct page titles/descriptions.
- ☐ Configure Open Graph and social sharing images.
- ☐ Configure robots.txt and sitemap.xml.
- ☐ Add structured data when appropriate.
- ☐ Use semantic HTML and descriptive headings.
- ☐ Verify HTTPS, redirects, domain, and production environment variables.
- ☐ Submit/index important pages with relevant webmaster tools where useful.
- ☐ Enable analytics and monitoring only with appropriate privacy controls.
- ☐ Identify a known-good rollback deployment.

Launch checklist

AREA	VERIFY
Domain	DNS, HTTPS, preferred host, redirects
Build	Production build passes
Environment	Required production variables present
SEO	Title, description, OG, sitemap, robots, structured data
Sharing	Social preview image/metadata render correctly
Quality	Responsive, accessibility, performance checks passed
Monitoring	Errors, uptime, analytics if needed
Rollback	Previous working deployment identified
Docs	README + operational notes current

Preview deployment flow

```
Local -> lint/typecheck/tests -> Git diff -> Preview deployment -> Smoke test -> Approve -> Production
```

14 CI/CD, Monitoring & Post-Launch Operations

As an app becomes important, the workflow should catch problems before users do and explain failures quickly.

- ☐ Automate lint, typecheck, tests, and production build in CI.
- ☐ Use preview deployments for meaningful changes.
- ☐ Track production errors and failed requests.
- ☐ Monitor uptime and key performance signals.
- ☐ Keep a clear rollback path.
- ☐ Review dependency updates and security advisories.
- ☐ Record important incidents and the fix.
- ☐ Measure user-facing outcomes, not only technical metrics.

Post-launch roadmap

WINDOW	FOCUS	TYPICAL WORK
Day 0-1	Stability	Watch errors, links, forms, logs, deployment health
Week 1	Real users	Fix friction, copy problems, mobile issues
Weeks 2-4	Quality	Accessibility, performance, SEO, cleanup

WINDOW	FOCUS	TYPICAL WORK
Month 2+	Growth	Content, conversion, integrations, experiments
Ongoing	Maintenance	Dependencies, security, backups, monitoring

15 Roadmap & Change Management

Simple product roadmap

```

NOW
- Current milestone

NEXT
- Highest-value follow-up

LATER
- Valuable but not urgent

NOT NOW
- Explicitly deferred ideas

```

Change-management checklist

- ☐ Create a small issue/goal before changing the product.
- ☐ Read the relevant docs first.
- ☐ Use the smallest safe implementation.
- ☐ Capture before/after screenshots for visual changes.
- ☐ Run lint, typecheck, tests, and build.
- ☐ Review the Git diff.
- ☐ Use focused commit messages.
- ☐ Update docs when behavior or architecture changes.
- ☐ Never mix unrelated refactors into feature work.

Professional branch/commit discipline

```

stable
  -> feature/[small-outcome]
      -> verify
      -> diff review
      -> commit
      -> preview
      -> merge

feat: add search
fix: handle empty results
style: refine hero spacing
perf: optimize image loading
chore: update dependency
docs: record architecture decision

```

16 What Not To Do

AVOID	WHY
-------	-----

AVOID	WHY
One giant “build everything” prompt	Hard to review; scope drifts; errors compound
Coding before defining scope	UI and architecture get rebuilt repeatedly
Overwriting the design reference	Visual consistency degrades
Hardcoding repeated data in JSX	Content becomes difficult to maintain
Making everything a Client Component	More JS and unnecessary complexity
Installing every UI/animation library	Bundle bloat and inconsistent patterns
Adding APIs before the UI needs them	Premature complexity
Global CSS hacks for single-feature behavior	Unintended side effects
Ignoring error/empty states	Real integrations fail eventually
Shipping without production-like QA	Works locally is not a release criterion
Over-abstracting early	More indirection makes AI and humans slower
Letting agents rewrite unrelated areas	Regression risk and hard-to-review diffs

17 Master A-Z Checklist

LETTER	ACTION	CHECKLIST
A	Aim	Define the problem and desired outcome.
B	Brief	Capture users, scope, constraints, references.
C	Create	Initialize the repo and clean baseline.
D	Design	Establish the visual source of truth.
E	Explain	Write PRD and architecture notes.
F	Framework	Choose and initialize the stack.
G	Guardrails	AGENTS.md, security, coding rules.
H	Helpers	Skills, UI primitives, shared utilities.
I	Implement	Build in small vertical slices.
J	Judge	Review against requirements and screenshots.

LETTER	ACTION	CHECKLIST
K	Keep	Separate content, UI, integrations, config.
L	Lint	Lint, typecheck, test, build.
M	Metadata	SEO, sharing, manifest, structured data.
N	Navigate	Routes, 404, loading/error states.
O	Observe	Logs, monitoring, analytics when useful.
P	Protect	Secrets, auth, validation, dependencies.
Q	Quality	Accessibility, responsive, performance.
R	Release	Deploy, domain, environment configuration.
S	Share	Verify social previews and indexing signals.
T	Test	Real devices, browsers, real flows.
U	Update	Keep docs and dependencies current.
V	Verify	Final diff and production smoke test.
W	Watch	Observe post-launch behavior.
X	eXclude	Remove dead code and unused dependencies.
Y	Yield	Collect feedback and prioritize.
Z	Zero-doubt	Ship only when acceptance checks pass.

18 Copy-Paste Prompt Library

A. Start a feature

```
Read AGENTS.md and the relevant docs first.
Implement only [feature].
Use existing components/data where possible.
Do not change [protected areas].
Acceptance criteria:
1. [observable behavior]
2. [responsive behavior]
3. [accessibility behavior]
Validation: run lint, typecheck, tests, build.
Stop after this feature; do not expand scope.
```

B. Visual correction

```
Compare the current implementation against the supplied design reference.
Fix only these visual differences: [list].
Preserve content, architecture, responsiveness, and interactions.
Do not redesign.
Check desktop/tablet/mobile and light/dark where applicable.
Capture a before/after screenshot.
Run lint + typecheck + build.
```

C. Debugging

```
Current behavior: [symptom].
Expected behavior: [expected].
Reproduce first. Inspect the relevant code and identify the root cause.
Apply the smallest fix.
Do not rewrite unrelated parts.
Verify the fix and run checks.
If the root cause is uncertain, report the evidence before making broad changes.
```

D. Safe refactor

```
Refactor only [target].
Preserve public behavior and visual output.
Prefer extraction/reuse over rewrites.
Do not change API/data contracts unless explicitly required.
Run tests/lint/typecheck/build and summarize the exact files changed.
```

E. Self-review

```
Review your implementation as a senior engineer.
Check for: regressions, accessibility issues, security risks, performance problems,
duplicated logic, unnecessary dependencies, scope creep, and unrelated file changes.
Fix only issues introduced by this task.
Then run the full validation commands.
```

F. Release check

```
Prepare this feature for release.
Verify production build, environment variables, metadata, responsive behavior, critical
flows, error states, accessibility, security, and rollback readiness.
Do not add new features.
Report PASS/FAIL for each check and list only blockers.
```

THE PROFESSIONAL HABIT After every meaningful batch: inspect -> run checks -> review the diff -> commit. Vibe coding becomes reliable when the feedback loop is fast, the context is explicit, and the scope stays small.